

Real-time image processing with a compact FPGA-based systolic architecture

César Torres-Huitzil*, Miguel Arias-Estrada

Computer Science Department, National Institute for Astrophysics, Optics and Electronics (INAOE), Apdo. Postal 51 & 216, Calle Luis Enrique Erro No. 1, Tonantzintla, Puebla, Mexico

Abstract

In this paper, a configurable systolic architecture on a chip for real-time window-based image processing is presented. The architecture was specially designed to implement efficiently, both in performance and hardware resource utilization, window-based image operators under real-time constraints. The computational core of the architecture is a configurable 2D systolic array of processing elements, which can provide throughputs over tenths of Giga Operations per Second (GOPs). The architecture employs a novel-addressing scheme that significantly reduces the memory access overhead and makes explicit the data parallelism at a low temporal storage cost. A specialized processing element, called Configurable Window Processor (CWP), was designed to cover a broad range of window-based image algorithms. The functionality of the CWPs can be modified through configuration registers according to a given application. For a current Field Programmable Gate Array (FPGA) prototype of a 7×7 systolic array, the architecture provides a throughput of 3.16 GOPs at a 60 MHz clock frequency. The processing time for a 7×7 generic window-based image operator on 512×512 gray-level images is 8.35 ms. The implemented window-based image operators include generic image convolution, gray-level image morphology and template matching. According to theoretical and experimental results, the architecture compares favorably with other dedicated architectures in terms of performance and hardware resource utilization.

© 2004 Elsevier Ltd. All rights reserved.

1. Introduction

Real-time image processing systems have gained an increasing importance in many fields, including applications such as: robotics, multimedia, virtual reality, industrial inspection, medical engineering, and autonomous navigation [1,2]. Most vision systems are commonly implemented on general-purpose processors receiving data from a camera attached to an analog to digital converter. However, general-purpose processors cannot always provide the computational power to fulfill real-time requirements due to their sequential nature, the large amount of data, and the heterogeneity of computations involved in image processing. To alleviate the inadequacy of sequential processors new

microprocessors have been developed to include advanced features with more than one processing unit, providing the ability to perform multiple operations in a clock cycle. An increased emphasis on cache design, data bus architectures and a multi-issue approach to issue multiple instructions in parallel, have provided means to improve microprocessor performances. Another way used to increase parallelism is using a single instruction, multiple data (SIMD) design. Typically, a processor with SIMD support can treat data in long registers as multiple smaller data words on which each processing element performs the same operation and generates multiple independent outputs as in the case of Pentium processors. The SIMD-like processing approach increases the speed of processor on vector-oriented algorithms, where the operations are inherently parallelizable [3].

However, the extent to which image processing applications are solved depends also on processing

*Corresponding author.

E-mail addresses: ctorres@inaoep.mx (C. Torres-Huitzil), ariasam@inaoep.mx (M. Arias-Estrada).

power, physical size, and power consumption of the vision systems involved [3]. The inherent data and instruction level parallelism of most data intensive or window-based image processing algorithms can be exploited to speed up the applications through the design of special purpose architectures on a chip according to a suitable parallel computational model [1]. The wide interest on data intensive or window-based image processing is due to the fact that more complex algorithms use the low-level results as primitives to pursue higher level goals.

Previously, there has been several hardware architectures reported in the literature to implement window-based image processing algorithms [4,5]. However, most of them focus on the performance of a single image processing algorithm, usually 2D image convolution, without considering implementation aspects such as flexibility, silicon area and power consumption.

In this paper, we propose an FPGA-based 2D systolic architecture as an alternative solution to implement window-based image processing algorithms on a single chip with a good performance-area tradeoff. The rest of the paper is organized as follows. Section 2 provides the mathematical basis of window-based image processing and discusses some hardware implementation details. Section 3 describes the proposed systolic architecture organization and the processing element structure. Also, a theoretical performance analysis of the architecture is provided in Section 3. Section 4 presents the FPGA implementation of the architecture, experimental results on performance and image results of some window-based image algorithms mapped to the architecture. Section 5 presents a performance analysis and comparison with other architectures and a discussion on the architectural aspects is provided. Finally, Section 6 presents the concluding remarks and further work.

2. Window-based image processing

2.1. Mathematical foundations

Several image processing applications involve the computation of a large number of repetitive operations over the entire input image in order to analyze the image contents and recover useful information. However, this task could be complex and computationally expensive for most conventional processors, as in the case of window-based image processing. Window-based image processing requires a large amount of low-level processing on massive amounts of data and the processing time depends on several factors, being of relevance the image and window mask sizes.

A special and common case in window-based image processing is the 2D convolution used for feature extraction and image filtering. In the simplest form of

a 2D convolution, the window operation function is an addition of pixels of a window in the input image weighted by the corresponding coefficients of the mask [6]. In mathematical terms, 2D image convolution can be expressed by the following equation:

$$Y[r, c] = \frac{1}{s} \sum_{i=0}^{w-1} \sum_{j=0}^{w-1} W[i, j] \cdot X[i + r, j + c], \quad (1)$$

where X is the input image, W is the $w \times w$ window mask, Y is the convolved output image and s is a scaling factor.

According to Eq. (1), two kinds of operations are required to compute a convolution. A scalar function operating on a pixel $X_{i+r,j+c}$ and a window coefficient $W_{i,j}$ to produce a partial result of the convolution, and a local reduction function to compute a single value from the partial results computed over the window domain using the scalar function. In Eq. (1), the multiplication represents the scalar function, and the accumulative addition represents the local reduction function.

Other image operators in image processing can be described in a similar way as the convolution case and they can be generalized as window-based image operators. For instance, in template matching [6], a template window $g[i, j]$ is given where it is required to find or locate its instances in a target image $f[i, j]$. Usually, the absolute difference between pixels of two images and the accumulation are used as the scalar function and the local reduction function, respectively.

Other image algorithms can be formulated as window-based operators such as 2D feature detection, gray-level image morphology, block matching and stereo disparity computation, among others. From the two examples discussed above, it is possible to extend the concepts of convolution and template matching toward the definition of a generic window-based image operator as follows.

A conceptual view of a window-based image operator is shown in Fig. 1. A window-based image operator is performed when a window with an area of $w \times w$ pixels is extracted from the input image and it is transformed according to a window mask or kernel, and mathematical functions to produce an output result [7]. The window mask is the same size as the image window and their values are constant through the entire image processing. The values used in the window mask depend on the specific type of features to be detected or recognized. Usually a single output data is produced by each window operation and it is stored in the corresponding central position of the window.

Window-based image operators can be formalized mathematically as follows. Let X be the $M \times N$ input image, Y the output image, and W a $w \times w$ window mask. A 2D window-based image operation can be

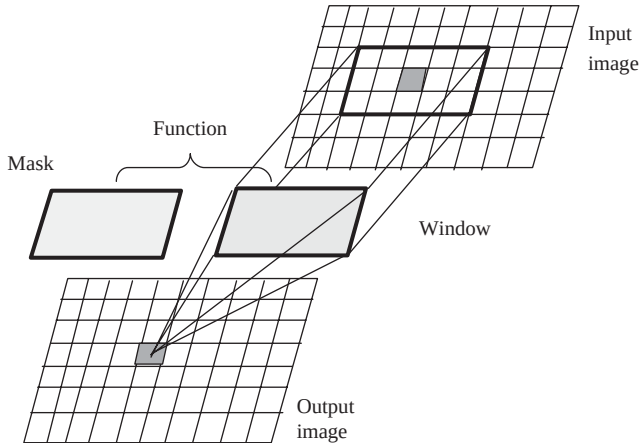


Fig. 1. A conceptual view of a generic window-based image operator on an input image.

defined according to the following equation:

$$Y_{rc} = F(f(w_{ij}, X_{r+i, c+j})) \quad \forall (i, j) \in w \times w, \quad \forall (r, c) \in M \times N, \quad (2)$$

where w_{ij} represents a coefficient from the window mask W , $X_{r+i, c+j}$ represents a pixel from a $w \times w$ data matrix around the (r, c) pixel in the input image, f defines the scalar function, and F defines the local reduction function.

According to Eq. (2), window-based image operators require three parameters to be completely specified [8]:

- A window mask: the size must be specified as well as the coefficient matrix.
- An elementary scalar function applied on a pixel-by-pixel scheme inside the window mask.
- A local reduction function to be applied to each extracted window of the input image to produce an output result.

The scalar function usually is part of a typical set of arithmetic-logic operations and they can be performed by an arithmetic component. A particular computation, such as addition, multiplication, or absolute value, is repeated on a sample-by-sample basis on a given image or between two images. A window-based operation is also based on a local reduction function. A local reduction function reduces the window of intermediate results to a single output result. In order to design flexible hardware architectures for window-based image processing, the architecture instruction set must support different scalar and local reduction functions.

2.2. Computational load

Window-based image operations are computationally expensive tasks and they constitute a significant computational load for most sequential processors.

Table 1

Computational load of image convolution expressed in terms of arithmetic operations

Elemental operations	Number of executions
Multiplication	$w^2 \times M \times N$
Addition	$(w^2 - 1) \times M \times N$
Load/store	$(2 \times w^2 + 1) \times M \times N$

The window-based operation complexity can be expressed in terms of the elementary arithmetic operations required to process an image [5]. In particular, analyzing the 2D image convolution, the total number of operations required to perform this operation, excluding address generation and other control tasks, is summarized in Table 1. The values presented in Table 1 were obtained considering the convolution of a $w \times w$ window mask on an $M \times N$ input image.

For instance, to convolve a 512×512 image with a 7×7 window mask over 50 million operations are required. In order to achieve real-time performance, i.e., 30 frames per second, a computational power of several GOPs is required. Thus, for a large-size window mask, the generalized convolution becomes even more computationally expensive because of the quadratic growth of w in the amount of computations. In general terms, a window-based operator has a computational complexity of $O(w^2 \times M \times N)$ for an $M \times N$ image with a $w \times w$ window mask.

2.3. Hardware implementation issues

The high data transfer rates and the computational load involved in window-based image processing require an efficient use of communication channel bandwidth and the use of parallel processing to achieve high processing efficiencies. Window-based image processing algorithms require the input image memory to be accessed several times. Considering the gap between processor speed and memory access time, the memory access time overhead is a critical issue to face. Although window-based image operations may access memory in different patterns, they share important features that can be generalized:

- Image operations are memory intensive and at least one new pixel of data is typically needed for each step in the computation.
- A high potential for parallelism is available since window operations include a large percentage of independent operations that are applied to each pixel.

However, the fact that image data is represented in linear memory organizations, so that neighboring pixels in the image are not necessarily stored as neighboring

elements in memories, obscures the spatial relationship between the pixels so the 2D data parallelism is not explicit. If the spatial data dependencies are exposed, it should be easier to implement a hardware architecture to use uniform two dimensional data access patterns that are needed for data parallel execution. A related issue is that data for window operators usually overlaps with the neighbor windows of the surrounding pixels. This means that there is a great deal to create simple vectors of data elements that can be processed by parallel vectorization techniques. In this work, a column based addressing mode is employed in order to exploit the data parallelism in window-based image operators [9].

3. Proposed hardware architecture

In this section the proposed hardware architecture for window-based image processing is presented. The architecture exploits data parallelism and data reuse through a column-based memory addressing mode and a set of processing elements working in parallel. The computation core of the architecture is organized under a 2D systolic array of processing elements. The processing element specially designed for the systolic architecture is called Configurable Window Processor (CWP).

The architecture is a stand-alone processor, but the memories for storage of the input image and the processed image are external to the architecture. The architecture reads data from the input memory banks where input image pixels and window mask coefficients are stored, denoted as P and W , respectively. The image pixels are read in a column-based scan and they are transmitted to the array of processing elements to compute, in parallel, several window operations.

3.1. Systolic array organization

Fig. 2 shows a block diagram of the 2D systolic organization of the CWPs employed in the architecture. The systolic array exploits the 2D parallelism through the concurrent computation of several window operations, i.e., the parallel computation of window operations through rows and columns in the input image. In Fig. 2, the module denoted by CWP represents a processing element, the module labelled by a D represents a delay line or shift register and the module labelled with a LDC represents a Local Data Collector. For each column of the systolic array there is a local data collector that collects results of CWPs located in the same column of the array. The global data collector module collects the results produced in the array and sends them to the output memory.

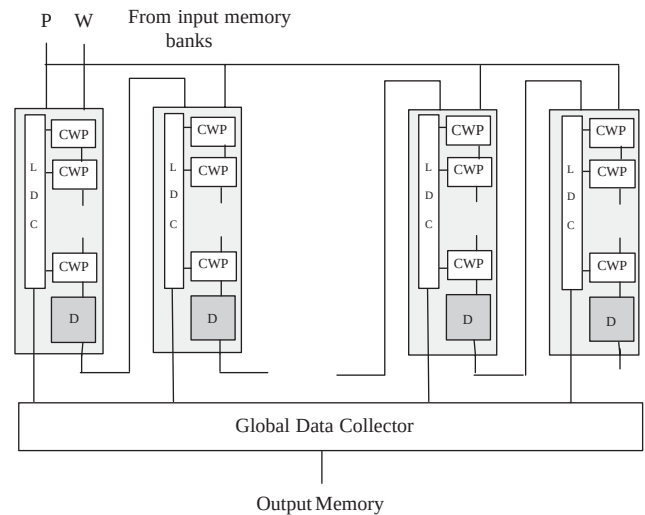


Fig. 2. Block diagram of the 2D systolic array of CWPs for window-based image processing. P represents pixel values of the input image and W the associated kernel weights.

The CWPs in the same column, enclosed by a rectangle, compute a window operator on the same column of the input image. This set of CWPs can be considered as a 1D systolic array. Data flows from top to bottom, and the CWPs work progressively in a systolic pipeline. A delay line is required in the boundary of the CWPs in a column in order to support window coefficient synchronization among columns of CWPs in the systolic array. The delay line is designed as a serial-input serial-output shift register. Thus, the window coefficients are temporally stored in the delay line, delayed for synchronization purposes, and then transmitted to the next column of CWPs. The stage number in the delay line depends on the loop unrolling degree employed, i.e., the number of rows processed in parallel. If N rows are processed in parallel, then a delay line with $N-1$ registers is required. The local data collector collects data from the CWPs, and the captured result is sent out of the CWPs to a global data collector. The local data collector scans progressively the CWPs from top to bottom according to a control signal from the control unit module in the architecture. The control module and the address generator unit are not shown in Fig. 3 for simplicity.

The interconnection of several columns of CWPs towards a 2D systolic array extends the concurrency of the architecture to 2D parallelism through parallel processing, pipeline and loop unrolling [5,9]. The interconnection between two contiguous columns of CWPs is straightforward and it can be considered a cascaded connection of 1D systolic arrays. In the 2D array, data flows from top to bottom of a column, and then left to right through columns.

As a whole, the architecture operation starts when a pixel from the input image is broadcast to all the

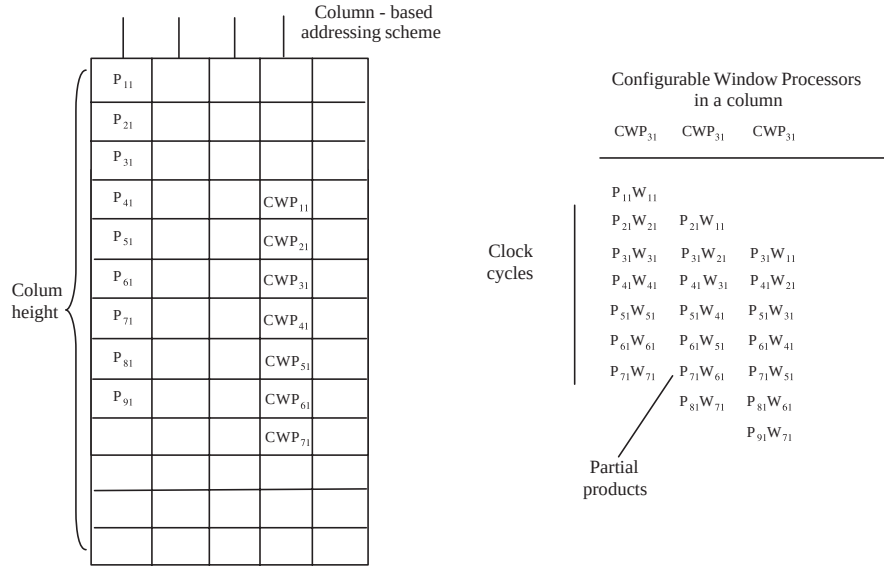


Fig. 3. Data flow and computation steps for three CWP in the same column of the systolic array. Each CWP performs the computation of a convolution operation.

processing elements in the array. Each CWP keeps track of a particular window-based operation. At each clock cycle, a CWP receives a different window coefficient, stored in an internal register, and an input image pixel that is common for all the CWPs. These values are used by each CWP to carry out a computation, specified by a scalar function, and to produce a partial result of the window operation. The partial results are incrementally sent to the local reduction function implemented in the CWP to produce a single result when all the pixels of the window are processed.

Fig. 3 shows the data flow and computation steps for image convolution for three CWP in the same column of a 7×7 systolic array. The considered window mask size is 7×7 and a window coefficient is denoted by W . The CWPs start to process progressively one after another and, on each clock cycle each CWP performs a multiplication between the current pixel with the corresponding window coefficient, and the result is accumulated with the previously computed value. Note that all the CWPs process the same input image pixel with a delayed window coefficient. The data flow can be generalized to a larger number of CWPs in the array.

After a short latency period, each CWP has progressively computed an output of a window operation. The produced results are captured by the data collectors and stored in the output image memory. Once a result produced by a CWP is collected, the CWP is ready to start a new computation. The CWPs work progressively in the same way until all data in the input image has been processed. The CWPs are continuously being reutilized for computing different windows in the input image.

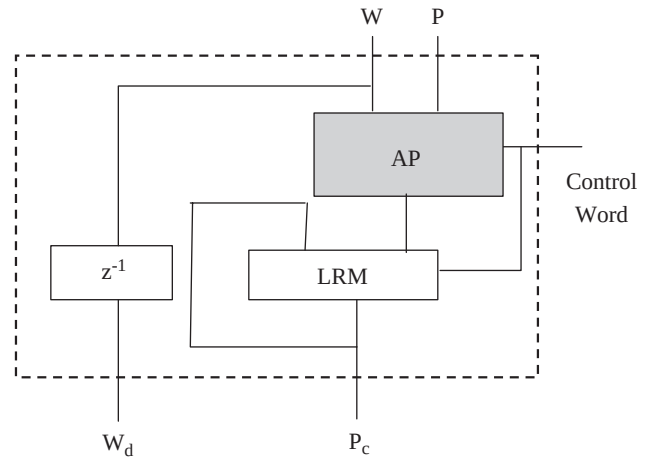


Fig. 4. Block diagram of the CWP, which includes an AP, a LRM, and a register.

3.2. Configurable window processor structure

Fig. 4 shows a simplified block diagram of the CWPs designed to cover most window-based operations in image processing. The CWP is composed of a register, an Arithmetic Processor (AP), and a Local Reduction Module (LRM). The AP is based on a similar scheme of an Arithmetic Logic Unit (ALU) and provides the hardware support for the scalar functions. It includes a multiplier, an adder/subtractor, and a distance computation module. The LRM module implements the local reduction function and it includes an accumulator and a maximum/minimum computation module.

The CWP is configurable by a control word selected by the end user. The CWP has two operational inputs, pixels from the input image and coefficients from the mask window, denoted by P and W , respectively. Each CWP has two output signals. One of them is the result of the window operation (P_C) and the other one is a delayed value of a window coefficient (W_d) that is transmitted to its neighbor CWP or to a delay line. The CWP accepts one pixel data and a window coefficient to compute one step of a window operation in a clock cycle.

On each clock cycle, each CWP executes in parallel three disjoint operations:

- Compute the pixel-by-pixel value to be passed to the next computation cycle.
- Integrate the contents of the output register, calculated at the previous clock cycle, with the new value produced by the AP module.
- Read a new window coefficient and store it into the register. Then, transmit the previous coefficient to the next CWP.

3.3. Performance analysis

The hardware architecture presented in the previous sections is dedicated to window-based image processing under real-time constraints. The main concerns for the architecture design were a regular input data scheme with a small number of data input/output ports, and fast parallel processing of window operations. In this section, the processing time for an input image in terms of architectural parameters is computed. Throughout the analysis, it is considered that the proposed architecture consists of $N_{CWP}(R)$ rows of CWPs. Each row includes $N_{CWP}(C)$ Configurable Window Processors.

The required processing time to process an input image with a window-based operator is composed of two main times. The *latency time* and the *parallel processing time*. The *latency time* corresponds to the time required to fill the pipeline inside the 2D systolic array. The latency time is measured between the activation of the first CWP until the activation of the last CWP. The latency is significant when the 2D systolic array starts the processing of a new set of rows of the input image.

Eq. (3) summarizes the latencies at the beginning of each row image, i.e., the overall time required to initialize full pipeline operations of the parallel modules at the beginning of row processing:

$$\tau_l = [(MaskSize - 1) \times (N_{CWP}(R) + MaskSize - 1)] \times \frac{1}{F} \frac{M}{N_{CWP}(R)}, \quad (3)$$

where $N_{CWP}(R)$ is the number of rows processed in parallel, $MaskSize$ is the size of mask used in the computations, F is the main clock frequency, M is the

number of rows of the image and N is the number of columns of the image.

The term enclosed in brackets in Eq. (3) represents the number of clock cycles required to read out the data in extended columns to fill the pipeline and to ensure all the CWPs are working concurrently. The factor $M/N_{CWP}(R)$ is related to the number of rows processed in parallel. Since several rows are processed in parallel, this term represents the number of strips in which the image is split into.

The *parallel processing time* is the time that all the CWPs in the 2D systolic array are working in parallel through the whole image processing. This is the addition of all the computing times to process a set of rows, without considering the latency at the start of processing. The required parallel processing time to process $M \times N$ pixels, without the initial latency is given by the following equation:

$$\tau_p = M \times N \times \left[\frac{N_{CWP}(R) + MaskSize - 1}{N_{CWP}(R)} \right] \frac{1}{F}. \quad (4)$$

The factor enclosed in brackets represents the average clock cycle number to produce a result. The overall time needed to process an $M \times N$ sized image is given by the addition of the preceding times:

$$T = \tau_l + \tau_p. \quad (5)$$

From the architectural point of view, the performance is compromised with the number of rows processed concurrently, hence with the silicon resources allocated to the architecture.

4. FPGA implementation and results

This section presents the FPGA synthesis results for the main modules of the architecture. To achieve the required architecture performance and configurability, a fully parameterizable and structural description of the modules of the proposed architecture was done using the hardware description language VHDL. The FPGA technology was chosen since it provides some implementation advantages such as [10,11]: (i) FPGAs provides massive parallel structures and high-density for logic arithmetic at relatively short design cycles, (ii) in FPGA devices, tasks are implemented by spatially composing primitive operators rather than temporally, (iii) in FPGAs, it is possible to control operations at bit level to build specialized datapaths, (iv) FPGA technology can offer potentially two orders of magnitude more raw computational power per unit of area than conventional processors, and (v) FPGA technology is well suited for implementing parallel architectures such as SIMD-like and systolic processors.

The modules and the complete architecture were synthesized with the Xilinx Synthesis Technology (XST)

tool and placed and routed with Foundation ISE 5.2i from Xilinx. All the simulations were carried out with Active-HDL, both functional and post-synthesis simulations.

4.1. FPGA synthesis

The digital synthesis of the architecture was performed in a hierarchical and incremental way targeted to an XCV2000E-6 VirtexE FPGA device. The results for the hardware resource utilization are summarized in Table 2, using as main parameters the number of slices, and specifically the number of flip-flops and look-up tables of the FPGA.

The results shown in Table 2 were obtained with the XST synthesis tool configured to optimize for speed. A 7×7 systolic array of CWP's was synthesized. Since no temporal restrictions of low-level were imposed on the place and route process further improvements in area and speed could be obtained if timing and area restrictions are imposed. The hardware resource utilization is about 30 percent, only a small part of the available FPGA resources. In fact, the architecture suits well for FPGAs with lower logic capacities. For instance, it is possible to synthesize the architecture in a XCV800 Virtex device, yielding a lower cost for the architecture. The FPGA resource utilization for this device is about 66 percent.

A summary of the technical data for the FPGA implementation of the current architecture prototype, highlighting aspects such as technology, hardware resources, and performance, is shown in Table 3.

4.2. Performance

In this section the results of the processing time of the proposed architecture for performing window-based operators on an input image are presented. In order to plot the performance graphs, the FPGA synthesis results and Eqs. (3) and (4) were employed. From synthesis results the most important factor to take into account is the maximum clock frequency.

The processing time for a generic window-based operator with a 7×7 window mask on 512×512 gray-level input images when mapped into the architecture is 8.35 ms. Equivalently, the proposed architecture is able to process about 120 512×512 gray-level images per second. Thus, the architecture provides more computational power than the required for the common accepted standard of 30 frames per second. At this point, it is important to notice the importance of reducing the processing time since an image processing application does not imply only the application of a window-based image operator but a sequence of different image operators.

Table 2

Hardware resource utilization for a 7×7 systolic array configuration on a XCV2000E Virtex device

Module	Number of slices	4-input LUTs	Number of flip-flops
CWP	101	190	24
Address generator unit	154	269	97
Control unit	5	10	3
A column of 7 CWP's	840	1506	223
Complete architecture	6118	10734	1604

Table 3

Technical data for the FPGA prototype for a 7×7 systolic array of the proposed hardware architecture

Feature	Specification
FPGA technology	0.18 μ m 6-layer metal process
Power supply	1.8 Volts
Number of processing elements, CWP's	49
Clock frequency	60 MHz
FPGA percentage for the complete architecture	6118 slices, around 30%
FPGA percentage per CWP	101 slices, less than 1%
FPGA percentage, address and control logic	159 slices, less than 1%
Number of flip flops	1604
Off-chip memory data buses	21-bit address, 8-bit data
On-chip memory	81 8-bit registers
Peak performance	3.16 GOPs
Number of pins	92

The processing time obtained for several families of FPGAs constructed with different technologies and for different parallelism degrees are plotted in Fig. 5. The obtained results were computed for a window mask of 7×7 on a 512×512 input image. Four different current high performance families were considered, the VirtexE, Virtex-II, Spartan-3, and the Virtex-II-Pro. Fig. 5 shows the processing time required to process the input image versus the number of rows processed in parallel. The clock frequencies reported by the synthesis tool were 60, 75, 90, and 92 MHz for the VirtexE, Virtex-II, Spartan-3, and the Virtex-II-Pro families, respectively.

As can be seen from Fig. 5, the FPGA implementation of the architecture for Spartan-3 and Virtex-II-Pro provides higher performances than the other families. This is due to the fact that these FPGA families include dedicated multipliers in their internal architectures. The curves for Spartan-3 and Virtex-II-Pro are almost overlapped since they operate almost at the same clock frequency. Another aspect to point out is that an increase in the number of processing elements produces

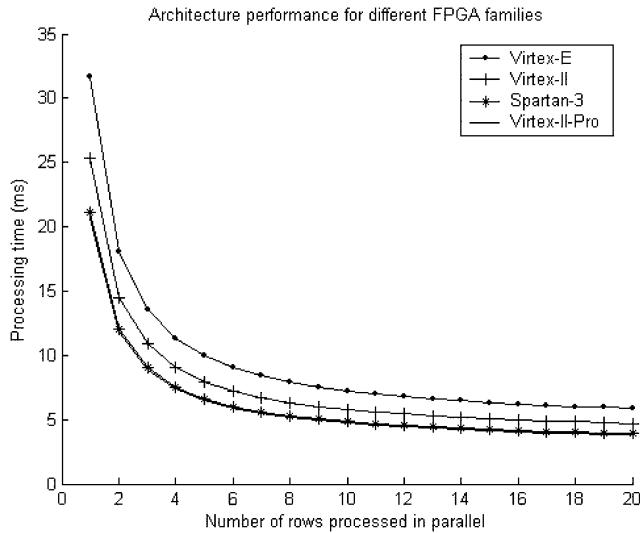


Fig. 5. Architecture performance for different FPGA families versus different parallelism degrees.

an increase in the overall performance of the architecture. According to the data presented in Fig. 5, it is possible to achieve real-time performance for video rate flows, the processing of 30 frames per second, with three to four rows processed in parallel.

4.3. Implemented algorithms

An available prototype of the architecture, synthesized for a XCV2000E Virtex device, executes approximately 3.16 GOPs at a 60 MHz clock frequency. The main characteristics of the architecture are expressed in terms of the following parameters: (i) variable window size, (ii) programmable window coefficients, (iii) configurable image size, (iv) configurable pixel word length, (vi) configurable functionality through a control word for different window-based image operators.

An experimental image processing set-up was developed to validate the architecture using an AlphaData FPGA board [12], centered on a XCV2000E Virtex device, and connected to a framegrabber. In order to map a window-based image operation into the hardware platform, the user, through the developed software interface, only has to establish the window coefficients and select the functionality of the architecture specifying a control word for a configuration register. This information is transmitted in different stages to the hardware platform through the software interface. After the architecture was configured, the user can send the image data and start the image processing.

Some window-based image algorithms were mapped into the systolic architecture and they are being integrated in a vision library as hardware building blocks. These algorithms will be used as basic blocks to build compound and more complex computer vision applications. Among the window-based image algo-

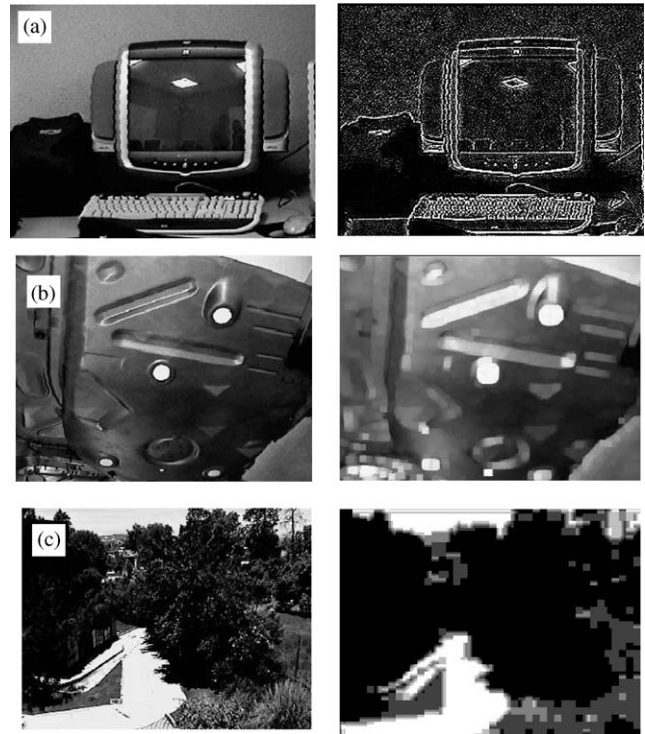


Fig. 6. Some input images and their corresponding output images for different window-based image algorithms for (a) Laplacian of a Gaussian, (b) gray-level dilation, and (c) template matching.

gorithms already mapped into and tested are: generic convolution, gray-level image morphology and template matching. Fig. 6 shows some input images and their corresponding output images for some common window-based image operators. Only the images processed by the architecture are presented for different algorithms since the software implementations produce the same image results. Fig. 6(a) shows an input image of a personal computer and its corresponding output image after applying a LoG filter. Fig. 6(b) shows a car chassis input image and its corresponding image after applying gray-level dilation. Finally, Fig. 6(c) shows an input image of a landscape and the output image produced by a 7×7 white rectangle template matching operation.

5. Analysis and discussion

In this section, the window-based operation performance on the proposed architecture is compared with other currently available architectural solutions. Comprehensively comparing image processing hardware architectures is a complex task for a single research effort. Moreover, the lack of standard metrics increases the complexity to do a reliable comparison. Thus, the standard metric of processing time will be used in the comparison.

Table 4

Performance comparison of the architecture with other proposed systems for window-based image processing

System	Architecture	Application	Image size	Timing
TMS320C80	Multiprocessor	5×5 Gaussian convolution	512×512	40 ms
TMS320C80	Multiprocessor	3×3 Gray-level dilation	512×512	32.7 ms
Splash 2	Multi-FPGA	3×3 Median Filter	512×512	27 ms
PDSP16488 40 MHz	Hardwired ASIC	8×8 Convolution	512×512	6.56 ms
PPIP	64×64 2D SIMD array	5×5 Laplacian filter	512×512	197 μ s
PPIP	64×64 2D SIMD array	5×5 Gaussian convolution	512×512	730 μ s
LSI Logic's L64240, 20 MHz	Hardwired ASIC	8×8 Convolution	512×512	13.11 ms
Blue Wave System PCI/C6200	DSP-based,	3×3 Convolution	512×512	7.2 ms
Alacron's AI-860	I860 processor	8×8 Convolution	512×512	66.1 ms
UWGSP5	DSP-based, MVP	3×3 Convolution	512×512	19 ms
IMAP Vision	SIMD, 256 PE	3×3 Convolution	256×256	0.65 ms
IMAP Vision	SIMD, 256 PE	3×3 Gray-level dilation	256×256	0.31 ms
DECChip 21064	Multiprocessor	5×5 Convolution	512×512	220 ms
MAP1000 200 MHz	Media Processor VLIW	7×7 Convolution	512×512	7.9 ms
VP24000/10	Vector computer	8×8 Template Matching	512×512	40 ms
Pentium IV, 1.5 GHz	Superscalar	7×7 Convolution	512×512	48 ms
Proposed architecture	Systolic	7×7 generic window-based image operator	512×512	8.35 ms

The timings for some window-based image operators and their corresponding implementations on different hardware architectures are summarized in Table 4. In Table 4, the window-based image application, the machine architecture, image size, and processing time are used as the main parameters for comparison [9,13,14].

The architecture performance is similar and even better than other architectures and systems reported in the literature, as shown in Table 4. Most of the architectures presented in Table 4 use a large number of processing elements and a considerable local storage compared to the proposed architecture. Moreover, some architectures can only perform convolutions, and they cannot accommodate a window mask larger than 8×8 . The processing time for the proposed architecture is even less than the execution time required by other ASICs architectures. Since the architecture is not implemented on an ASIC, its performance regarding to these ASIC architectures is significant. If the architecture or some critical parts were mapped to an ASIC an improvement on the clock frequency could be obtained and hence a better performance.

The architecture provides a high throughput, 3.16 GOPs and a fixed processing time independent of the window-based algorithm mapped to the architecture. The estimated computational power for the Intel Pentium IV processor is approximately 1 GFlop. It implies that the Pentium IV provides less computational density [11] than the proposed architecture when implementing window-based image applications. However, it is important to point out that more work needs to be done on the improvement of the CWP utilization. According to the results presented in Fig. 5, the utilization degree of the CWPs is dependent on the

number of CWPs per column. In the current topology of the architecture, if the number of CWPs increases the utilization degree of each CWP decreases. A mechanism for reutilization of CWPs has to be explored to enable the CWPs to process more than one window-based operation in the same column. As a consequence, the number of CWPs per column can be decreased without scarifying computational power but at the cost of more complicated control logic and an increase in the local storage per CWP.

One of the main advantages of the proposed architecture is its configurability. The architecture is flexible enough to support several variations of window-based image operators. Thus, the architecture offers the possibility of addressing different image-based applications from a System on a Chip (SoC) perspective. In order to analyze the requirements of area for different configurations of the architecture, a graph of the number of FPGA hardware resources used in the FPGA versus a particular window-based application was obtained. As shown in Fig. 7, the hardware resources, number of LUTs, slices, and flip-flops in the FPGA depends on a particular window-based image application mapped into the FPGA. The most hardware demanding application is the convolution since it requires a multiplier per each CWP in the array to perform the operation.

From Fig. 7, it can be shown that it is possible to achieve real-time performance for window-based image processing with very compact architectures. The low resource utilization allows the architecture to be implemented in compact mobile applications restricted to low-power consumption. Moreover, due to the compactness and configurability of the architecture it is possible to explore partial reconfiguration in the

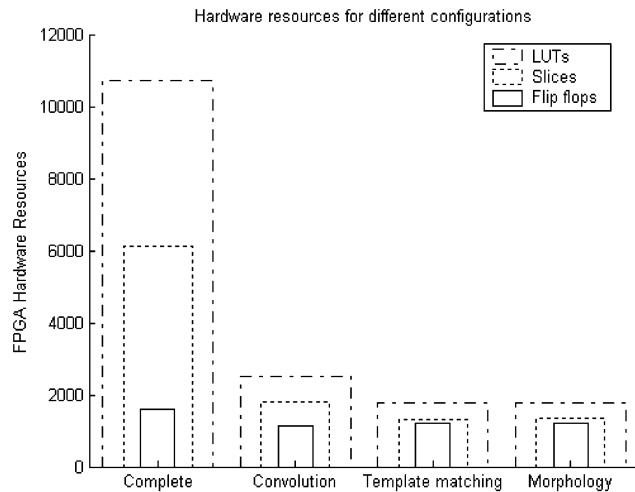


Fig. 7. FPGA hardware resources for different architecture configurations.

FPGA in order to use more efficiently the hardware resources and to match the application requirements at a given time.

Due to its versatile, modular and scalable design, the architecture constitutes a platform to explore more complex algorithms such as block matching for motion estimation, stereo disparity computation, and discrete cosine transformation, among others. These algorithms can be reformulated as window-based image algorithms. Some of these potential applications are currently being developed and mapped into the architecture. The overall architecture organization would remain almost unchanged and slight modifications for the address generator module and the control logic would be required.

6. Conclusions

This paper has presented a flexible FPGA-based hardware architecture for real-time window-based image processing. A set of processors organized under a systolic approach constitutes the computational core of the architecture. The current FPGA-based architecture prototype can achieve a processing rate of near 3.16 GOPs with a 60 MHz clock frequency. The high efficient and compact hardware architecture opens new and practical possibilities to mobile machine vision systems where size and power consumption are hard constraints to achieve. The architecture deals with a broad class of image processing algorithms and the configurability of the proposed architecture provides the additional advantage of being more flexible than the ASICs based on hardwired approaches. The CWP developed in this research can be used and adapted to different algorithms based on a window processing scheme. The explored window-based image processing algorithms includes

generic convolution, template matching and gray-level image morphology. Moreover, some other applications suits well for the architecture such as block matching for motion estimation, stereo disparity computation, among others.

The performance comparison with other existing architectures confirms the promising advantages of the proposed FPGA-based systolic architecture over other conventional approaches. Its performance has been evaluated for many window-based target applications with excellent results that validate the proposed high performance architectural model.

The architecture processing capacity can be improved if some extensions are analyzed. For instance, the use of wider buses for the image data and image buffers, a reutilization scheme for CWPs through columns in order to increase the utilization degree, the inclusion of local storage inside the CWPs in order to process image using larger windows or more irregular algorithms, the architecture coupling to a microprocessor to support more irregular operations and for decision making in the same chip. Moreover, the computational density of the architecture can be improved if aspects of dynamic reconfiguration are explored. Reconfigurability provides the advantage to change the functionality in response to specific computational requirements of an application without significantly degrading performance as it is analyzed and discussed in [15]. In this sense, a high efficiency and compact architecture can be generated for any particular window-based algorithm based on the reuse of hardware resources. The execution scheduling of different algorithms could be exploited to build more complex applications with the same hardware resources.

The proposed architecture is well suited to be the computational core of a smart camera or a completely self-contained vision system since it provides a high computational efficiency, with moderate hardware resource utilization and power consumption. The smart cameras are an interesting and promising application for embedded system research.

Acknowledgments

The authors want to acknowledge the support received from Conacyt through the Project J32151-A. Reconfigurable Computing for Computer Vision, and the software and hardware support received from Xilinx through the Xilinx University Program.

References

- [1] Ranganathan N.. VLSI & parallel computing for pattern recognition & artificial intelligence. Series in Machine Perception and Artificial Intelligence, vol. 18. Singapore: World Scientific Publishing; 1995.

- [2] Laplante PA, Stoyenko AD. Real-time imaging: theory, techniques, and applications. New York: IEEE Press; 1996.
- [3] Silc J, Ungerer T, Robic B. A survey of new research directions in microprocessors. *Microprocessor and Microsystems* 2000;34:175–90.
- [4] Bosi B, Bois G, Savaria Y. Reconfigurable pipelined 2-D convolvers for fast digital signal processing. *IEEE Transactions on VLSI* 1999;7(3):299–308.
- [5] Managuli R, York G, Kim D, Kim Y. Mapping of two-dimensional convolution on very long instruction word media processors for real-time performance. *Journal of electronic Imaging* 2000;9(3):327–35.
- [6] Jain R, Kasturi R, Shunck BG. *Machine vision*. New York: MacGraw-Hill; 1995.
- [7] Li D, Jiang L, Kunieda H. Design optimization of VLSI array processor architecture for window image processing. *IEICE Transactions on Fundamentals* 1999;E82-A(8):1474–84.
- [8] Bouridane A, et al. A high level FPGA-based abstract machine for image processing. *Journal of Systems Architecture* 1999; 45(10):809–24.
- [9] Arias-Estrada M, Torres-Huitzil C. Real-time field programmable gate array architecture for computer vision. *Journal of Electronic Imaging* 2001;10(1):289–96.
- [10] Marcahal P. Field programmable gate arrays. *Communications of the ACM* 1999;42(4):57–9.
- [11] DeHon A. The density advantage of configurable computing. *Computer* 2000; 41–49.
- [12] Sweeney C, Blyth B. RC1000-PP hardware reference manual. Celoxica Limited, Version 1.22, 2000.
- [13] Singh H, et al. MorphoSys: an integrated reconfigurable system for data-parallel and computation-intensive applications. *IEEE Transactions on Computer* 2000;49(5):465–81.
- [14] Lohier F. *Méthodologies de programmation et évaluation des processeurs de traitement de signal parallèles pour le traitement d'images en temps réel*, PhD dissertation, L'Université Pierre Et Marie Curie Paris 6, February 2000.
- [15] Kessal L, Abel N, Demigny D. Real-time image processing with dynamically reconfigurable architecture. *Journal on Real-time Imaging* 2003;9:297–313.